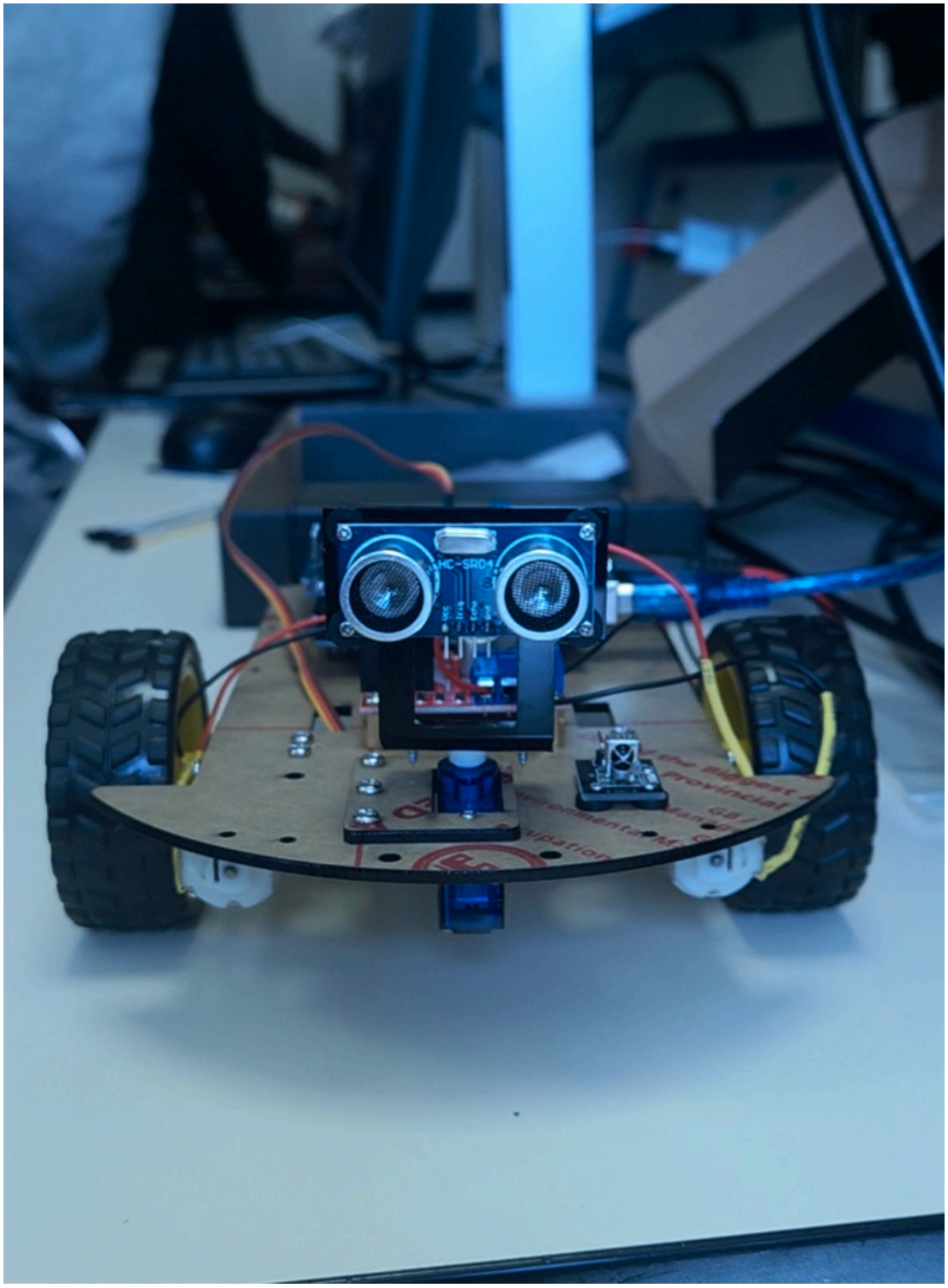
Encadré par Monsieur Traore

SAE ROBOT



Réalisée par : Sirine Allout, Abrar Mathlouthi et Awa Soumare.

Introduction :

Aujourd'hui, la technologie ne cesse de progresser et l'automatisation s'impose dans de nombreux domaines pour optimiser les productions et faciliter les tâches. Dans l'industrie, par exemple, de nombreuses machines automatisées sont utilisées afin d'augmenter la productivité tout en réduisant la pénibilité du travail humain.

Dans le secteur automobile, on retrouve des bras robotisés suiveurs de ligne équipés de capteurs à ultrasons. Leur rôle est de déplacer des pièces lourdes (comme des échappements ou des éléments de carrosserie) d'un point A à un point B, limitant ainsi les risques de blessure pour les ouvriers.

Dans le cadre de cette SAE, notre objectif était de concevoir, programmer et tester un robot mobile autonome à l'aide d'une carte Arduino. Équipé d'un capteur à ultrasons, ce robot devait être capable de détecter les obstacles et d'adapter sa trajectoire. Pour mener à bien ce projet, nous avons utilisé un diagramme de Gantt afin de nous organiser et de répartir les rôles au fil des séances.

SOMMAIRE

1. Cahier des charges

2. Prise en main du matériel

3. Montage du robot

4. Installation du logiciel Arduino

5. Ajout des bibliothèques

6. Programmation des actions du robot

7. Code final

8. Difficultés rencontrées durant le SAE

9. Conclusion

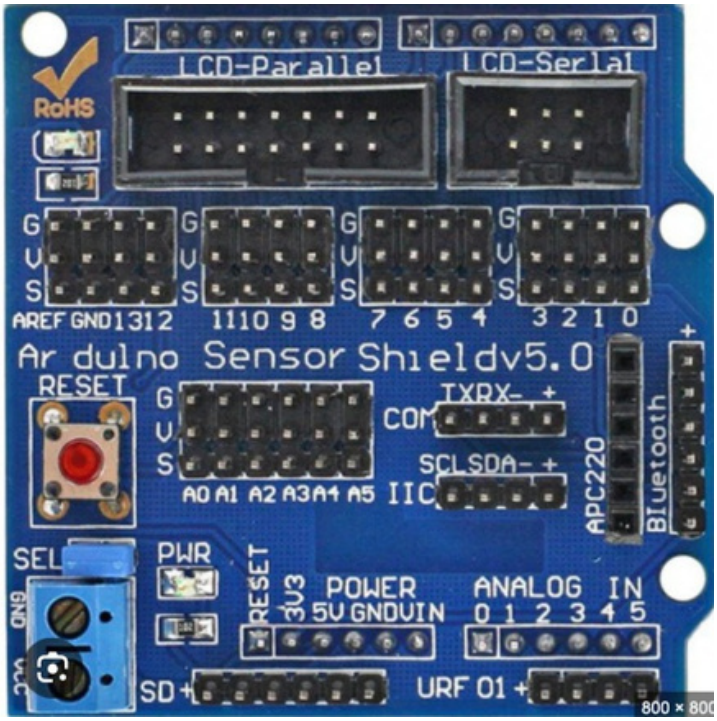
1. Cahier des charges

Chaque membre du groupe a eu un rôle à jouer dans ce projet :

Prénom	Rôle	Explication
Abrar Mathlouthi	Animateur	Fait vivre le groupe veille à la participation de chacun et au bon déroulement des échanges
Awa Soumare	Chef de projet	Organise le projet, répartit les tâches, donne les décisions après dissertation avec le groupe.
Sirine Allout	Secrétaire	Chargé de la prise de notes pendant les projet, rédaction des comptes rendus

•

2 . Découverte du matériel et vérification

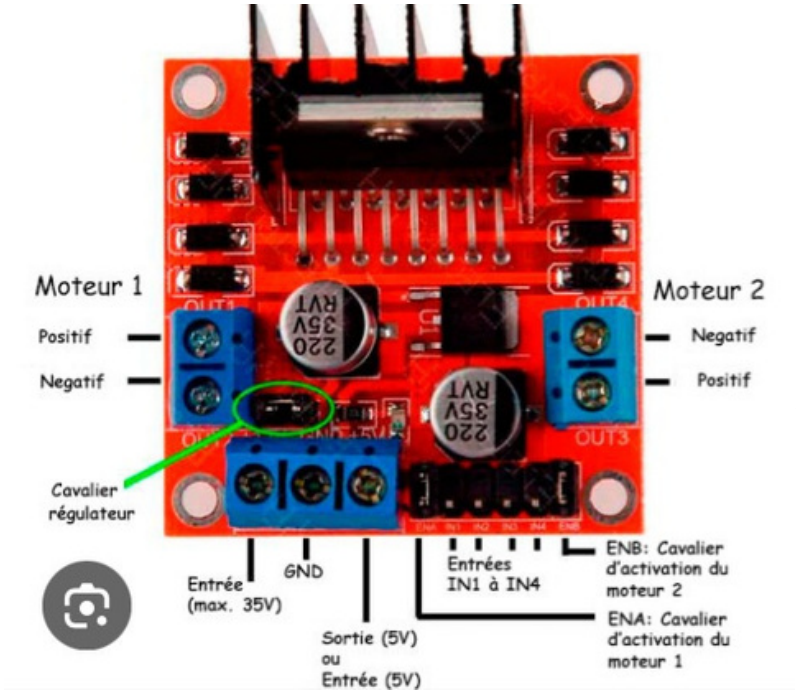


V5 Expansion Board

Il s'agit d'une carte d'extension qui simplifie les liaisons entre l'Arduino UNO et les divers éléments du robot. On y trouve des ports pour les moteurs, des capteurs ultrasonique, un module infrarouge, un cerveau moteur et des points d'alimentation cette carte offre aussi la possibilité de fournir de l'énergie au robot directement par le biais d'une batterie ou d'une source externe, tout en sauvegardant l'Arduino.

Carte Arduino UNO

C'est la carte principale du projet. Elle reçoit les informations envoyées par les capteurs (comme les mesures de distance ou les signaux infrarouges.) et comment les moteurs en fonction du programme qui a été chargé.



Module moteur (L298N)

Il est utilisé pour contrôler les moteurs du robot. Il contient deux ponts en H, chacun composé de quatre interrupteurs (ou transistors) permettant de gérer un moteur. En activant ces interrupteurs selon une certaine séquence, on peut inverser le sens du courant dans le moteur, ce qui permet de changer sa direction de rotation. Ainsi, le robot peut avancer ou reculer.
De plus, le L298N permet de contrôler la vitesse des moteurs en utilisant un signal PWM (modulation de largeur d'impulsion).

Capteur à ultrasons HC-SR04

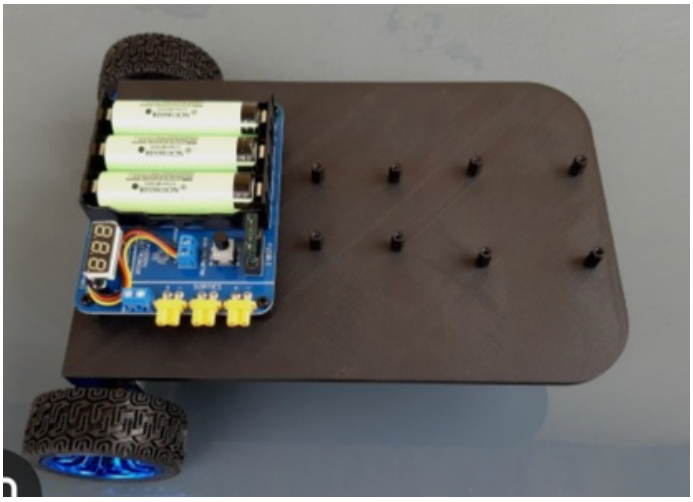
Il est utilisé pour mesurer la distance entre le robot et un obstacle. Il envoie des ultrasons à l'aide d'un émetteur, puis capte l'écho avec un récepteur. En mesurant le temps que met l'onde à revenir, il peut calculer la distance avec précision ainsi il peut éviter les obstacles en adaptant sa trajectoire.

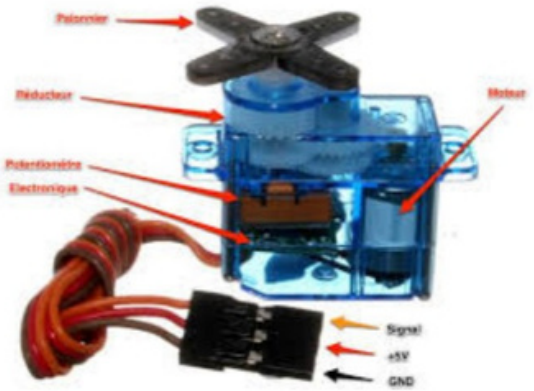
Télécommande infrarouge

Elle permet de contrôler le robot à distance en envoyant des signaux lumineux invisibles à l'œil nu. Ces signaux sont reçus par un capteur infrarouge (IR receiver module) placé sur le robot. Ce capteur décode les informations envoyées par la télécommande (comme avancer, reculer, tourner...).

Châssis

Il s'agit de la plateforme du robot sur laquelle tous les éléments sont fixés.





Servomoteur (Sg90)

Il est placé sous le captera ultrason. Il permet de réaliser des mouvements précis sur un angle limité à 180°.

DC motor

il est utilisé pour faire tourner les roues grâce à la commande envoyer par la carte L298N ce qui permet au robot de se déplacer dans différentes directions. Sa

Pile

Assure l'alimentation en électricité pour l'ensemble du système

Roue universelle

Elle rend le robot asile et mobile en facilitant ses déplacements dans toutes les directions

PARTIE 2: Montage du Robot

Pour respecter les instructions de montage, nous avons utilisée le guide qui nous a été fourni.

1- Montage du châssis

Nous avons fixé les deux moteurs à courant continu sur la plate-forme à l'aide de devis, ajouter les roues puis mis en place la roulette libre à l'arrière pour assurer la stabilité du robot.

2. Installer le servomoteur SG90

Le servomoteur SG90 a été mis en place à l'avant du robot, afin de permettre la rotation du capteur à ultrason.

3. Installer le capteur à ultrasons

Nous avons monté le capteur HC-SR04 directement sur le servomoteur, afin qu'il soit en mesure de faire pivoter et d'explorer l'environnement.

4. Mise en place de la carte Arduino Nous avons installé et sécurisé la carte

Arduino UNO sur la plaque supérieure.

5. Installer le module L298N Le module de commande de moteur (L298N) a été placé à

proximité de l'Arduino. Nous avons connecté les câbles des moteurs à courant continu aux

sorties du L298N
(OUT1 à OUT4).

6. Mise en place de l'alimentation

Nous avons installé un porte-billes sur le robot en vue d'alimenter le système.

7. Ranger les *fi*ls de manière ordonnée

Nous avons organisé tous les fils entre les éléments et les avons rassemblés.

8. Monter les deux niveaux. Suite à l'organisation des câbles, nous avons été contraints de fixer le deuxième étage pour des raisons de facilité.

2- Installation Arduino IDE

Nous utilisons l'IDE Arduino, un environnement de programmation, pour coder notre robot à l'aide de la carte Arduino UNO. Il permet de créer du code en Arduino, de le compiler et de l'envoyer.

Via un câble USB, directement sur la carte. L'IDE permet de concevoir et d'expérimenter différentes fonctionnalités du robot, comme le contrôle des moteurs, la détection d'obstacles à l'aide de capteurs ultrasoniques ou infrarouges, ainsi que la réception de commandes via une télécommande. L'IDE propose également l'option d'intégrer des bibliothèques additionnelles nécessaires pour certains composants. Il s'agit donc d'un instrument indispensable pour le développement, la mise à jour et l'opérationnalité maximale du robot dans ce projet.

3 . Ajout De la bibliothèque

Les bibliothèques Arduino sont des outils qui permettent d'ajouter facilement de nouvelles fonctions dans les programmes créés avec l'Arduino IDE. Elles contiennent du code déjà prêt, ce qui simplifie l'utilisation de certains composants comme les servomoteurs ou le capteur infrarouge. Dans notre projet, ces bibliothèques sont essentielles pour faire fonctionner correctement le robot avec les différents éléments électroniques.

4 . Programmation des codes du robot

Pour nous permettre de contrôler le **servomoteur** nous avons utilisé ce code :

sketch_may31a.ino

```
#include <Servo.h>  // On inclut la bibliothèque Servo pour pouvoir contrôler un servomoteur

// On crée un objet "myservo" de type Servo, qui permettra de commander le servomoteur
Servo myservo;

// On déclare une variable entière "pos" pour stocker la position actuelle du servomoteur (en degrés)
int pos = 0;

void setup() {
  // On attache le servomoteur à la broche A2 de la carte Arduino
  // Cela signifie que le signal de commande du servo est envoyé via la broche A2
  myservo.attach(A2);
}

void loop() {
  // Cette boucle fait tourner le servomoteur de 0 à 180 degrés, un degré à la fois
  for (pos = 0; pos <= 180; pos += 1) {
    // On envoie la position actuelle au servomoteur
    myservo.write(pos);

    // On attend 15 millisecondes pour laisser le temps au servo d'atteindre la position demandée
    delay(15);
  }

  // Cette boucle fait revenir le servomoteur de 180 à 0 degrés, un degré à la fois
  for (pos = 180; pos >= 0; pos -= 1) {
    // On envoie la position actuelle au servomoteur
    myservo.write(pos);

    // On attend 15 millisecondes pour permettre au servomoteur de se déplacer correctement
    delay(15);
  }
```

Nous l'avons commenté pour bien comprendre chaque ligne du code.

Ce programme est conçu pour piloter un servomoteur de 0° à 180°, puis revenir de 180° à 0°, tout en assurant une continuité et une précision constantes. Tout d'abord, l'intégration de la bibliothèque Servo.h est réalisée afin de faciliter l'usage des fonctions associées au contrôle d'un servomoteur. Ensuite, on crée une instance appelée my servo, qui sera employée pour transmettre les instructions au servomoteur. Dans la fonction setup(), on établit une connexion entre le servomoteur et la broche A2 de l'Arduino, ce qui indique que cette broche particulière sera utilisée pour transmettre les instructions. La fonction loop() intègre deux boucles for. La première opération consiste à faire tourner le servomoteur de 0 à 180 degrés, par paliers d'un degré à chaque fois, en marquant une brève pause de 15 millisecondes entre chacun des déplacements.
La seconde boucle réalise le mouvement inverse, allant de 180 à 0 degrés.

Pour nous permettre de contrôler le **capteur ultrason**, nous avons utilisé le code ci dessous:

sketch_may31b.ino

```
// Définition des broches utilisées pour le capteur à ultrasons
#define TRIG_PIN A1  // Broche A1 connectée à la broche TRIG du capteur
#define ECHO_PIN A0  // Broche A0 connectée à la broche ECHO du capteur

int Ult_distance = 0;  // Variable pour stocker la distance mesurée

// Fonction qui mesure la distance à l'aide du capteur ultrasonique
float checkdistance() {
  digitalWrite(TRIG_PIN, LOW);          // On s'assure que TRIG est bas
  delayMicroseconds(2);                 // Petite pause de 2 microsecondes
  digitalWrite(TRIG_PIN, HIGH);         // Envoi d'une impulsion de 10 µs
  delayMicroseconds(10);                // pour déclencher l'émission ultrasonique
  digitalWrite(TRIG_PIN, LOW);          // On remet TRIG à l'état bas

  // pulseIn mesure la durée de l'impulsion reçue sur ECHO
  // On divise par 58 pour convertir le temps en distance (en cm)
  float distance = pulseIn(ECHO_PIN, HIGH) / 58.00;

  delay(10);                            // Petite pause avant la prochaine mesure
  return distance;                      // On retourne la distance mesurée
}

void setup() {
  Serial.begin(9600);                   // Initialisation de la communication série à 9600 bauds
  pinMode(TRIG_PIN, OUTPUT);            // La broche TRIG est une sortie
  pinMode(ECHO_PIN, INPUT);            // La broche ECHO est une entrée
}

void loop() {
  Ult_distance = checkdistance();       // On appelle la fonction de mesure et stocke le résultat
  Serial.print("Distance: ");           // Affiche le texte "Distance: "
  Serial.print(Ult_distance);           // Affiche la distance mesurée
  Serial.println("CM");                 // Affiche l'unité (centimètres) et passe à la ligne
  delay(100);                           // Attente de 100 ms avant la prochaine mesure
}
```

Nous l'avons commenté pour bien comprendre chaque ligne du code

Ce code est utilisé pour mesurer la distance entre le robot et un obstacle à l'aide du capteur ultrasonique HC-SR04. Initialement, les broches TRIG (A1) et ECHO (A0) sont définies pour l'émission et la réception du signal. La fonction checkdistance()
représente le cœur du programme : elle initie l'émission d'ultrasons en transmettant
une impulsion de 10 microsecondes sur TRIG. Le capteur attend le retour de ses
ultrasons qui, après avoir rencontré un obstacle, lui seraient renvoyés.Actuellement,
la broche ECHO est configurée en mode HAUT, et la méthode pulseIn() mesure le temps nécessaire pour que le signal fasse un aller-retour.On procède ensuite à la division de cette durée par 58 pour déterminer la distance en centimètres, sachant que les ondes ultrasoniques parcourent environ 1 cm en 58 microsecondes (aller- retour).

Pour nous permettre de contrôler le **récepteur infrarouge**, nous avons utilisé le code ci dessous:

```
#include <IRremote.h>
int RECV_PIN = 12;
IRrecv irrecv(RECV_PIN);
decode_results results;
#define IR_Go       0x00ff629d
#define IR_Back     0x00ffa857
#define IR_Left     0x00ff22dd
#define IR_Right    0x00ffc23d
#define IR_Stop     0x00ff02fd
#define IR_ESC      0x00ff52ad
#define Lpwm_pin  5      //adjusting speed
#define Rpwm_pin  6     //adjusting speed //
int pinLB=2;            // defining pin2 left rear
int pinLF=4;            // defining pin4 left front
int pinRB=7;            // defining pin7 right rear
int pinRF=8;            // defining pin8 right front

void go_forward(unsigned char speed_val)    // speed_val: 0-
    {digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,LOW);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,LOW);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);
```

Cette partie du code commence par l'importation de la bibliothèque IRremote.h, permettant à l'Arduino de recevoir des signaux d'une télécommande infrarouge.

On branche le récepteur infrarouge à la broche 12 (RECV_PIN). On crée une instance de l'objet irrecv pour gérer la réception, pendant qu'un objet results est employé pour stocker les codes reçus. Ensuite, chaque bouton de la télécommande est lié à une valeur hexadécimale grâce à l'utilisation de #define : par exemple, IR_Go pour avancer, IR_Back pour reculer, et ainsi de suite. Nous passons ensuite à la détermination des broches de contrôle des moteurs et des broches PWM (Lpwm_pin pour le moteur gauche et Rpwm_pin pour le moteur droit).

Pour nous permettre de contrôler **la fonction de mouvement** , nous avons utilisé le code ci dessous:

```
void go_forward(unsigned char speed_val)    // speed_val: 0~255
    {digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,LOW);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,LOW);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);

    }

void go_backward(unsigned char speed_val)    // speed_val: 0~255
    {
     digitalWrite(pinRB,LOW);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,LOW);
     digitalWrite(pinLF,HIGH);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);
    }
```

```
void rotate_left(unsigned char speed_val)        // speed_val: 0~255
    {digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,LOW );
     digitalWrite(pinLB,LOW);
     digitalWrite(pinLF,HIGH);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);


    }
void rotate_right(unsigned char speed_val)    // speed_val: 0~255
    {
      digitalWrite(pinRB,LOW);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,LOW);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);

    }
void stopp()        //stop
    {
     digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,HIGH);
    }
```

Cette gamme de fonctionnalités permet de déterminer les différents mouvements que le robot peut effectuer, en ce qui concerne la direction de rotation de chaque moteur et la vitesse mise en œuvre via un signal PWM. La méthode go_forward(unsigned char speed_val) permet au robot de progresser en avant en contrôlant les moteurs latéraux gauche et droit, tout en attribuant une vitesse identique (qui peut aller de 0 à 255) grâce aux connexions PWM Lpwm_pin et Rpwm_pin.
La fonction go_backward() modifie les polarités des moteurs pour autoriser le robot à reculer à la même vitesse. Les fonctions rotate_left() et rotate_right() permettent de faire tourner le robot sur son axe, vers la gauche (en faisant bouger les roues dans des directions opposées) et vers la droite.
En conclusion, la fonction stopp() active les quatre broches de direction en mode haut (HIGH), ce qui provoque le freinage des moteurs et, par conséquent, l'arrêt immédiat du robot. Ces caractéristiques donnent au robot la capacité de se mouvoir avec exactitude en obéissant aux commandes reçues (par le biais d'une télécommande ou de capteurs), tout en modifiant simultanément son orientation et sa vélocité.

Pour nous permettre de contrôler la **Fonction IR_control**, nous avons utilisé le code ci dessous:

```
void IR_Control(void)
{
   unsigned long Key;
   if(irrecv.decode(&results)) //judging if serial port receives data
 {
     Key = results.value;
    switch(Key)
     {
       case IR_Go:go_forward(150);   //UP
       break;
       case IR_Back:go_backward(150);   //back
       break;
       case IR_Left:rotate_left(100);   //Left
       break;
       case IR_Right:rotate_right(100); //Righ
       break;
       case IR_Stop:stopp();   //stop
       break;
       default:
       break;
     }
     irrecv.resume(); // Receive the next value
    }

}
```

La fonction IR_Control() permet de manœuvrer le robot en utilisant une télécommande infrarouge. Lorsqu'un bouton de la télécommande est pressé, un signal infrarouge est émis et capté par un récepteur. Connecté à l'Arduino. On utilise la fonction irrecv.decode(&results) pour déchiffrer ce signal, et la valeur extraite est stockée dans la variable Key. Un switch-case évalue cette valeur en fonction des différentes correspondances des boutons de la télécommande. Selon le bouton pressé, l'action adéquate s'exécute : La fonction go_forward(150) déplace le robot en avant, go_backward(150) le fait reculer, rotate_left(100) assure une rotation à gauche, rotate_right(100) garantit une rotation à droite tandis que stopp() arrête son mouvement.

Pour nous permettre de contrôler la **Fonction setup()**, on a utilisé le code ci dessous:

```
void setup()
{
   pinMode(pinLB,OUTPUT); // pin2
   pinMode(pinLF,OUTPUT); // pin4
   pinMode(pinRB,OUTPUT); // pin7
   pinMode(pinRF,OUTPUT); // pin8
   pinMode(Lpwm_pin,OUTPUT); // pin5 (PWM)
   pinMode(Rpwm_pin,OUTPUT); // pin6 (PWM)
   irrecv.enableIRIn(); // Start the receiver
   Serial.begin(9600);   //initializing serial port, Bluetooth used as serial port, setting baud ratio at 9600
   stopp();
}
```

Dans la fonction setup(), chaque broche est configurée en mode sortie, car elles transmettent des signaux à la carte de contrôle du moteur. On appelle ensuite la fonction irrecv.enableIRIn() pour activer le récepteur infrarouge. La communication série est activée (Serial.begin(9600)). afin de faciliter le débogage ou l'affichage d'informations. Finalement, la fonction stopp() est utilisée pour immobiliser le robot au démarrage.

Pour nous permettre de contrôler la **fonction module de moteur**, nous avons utilisé le code ci-dessous :

En raison de la longueur du code nous y sommes allés partie par partie :

```
void loop()
{go_forward(100);
 delay(2000);
 go_backward(100);
 delay(2000);
 rotate_left(150);
 delay(2000);
 rotate_right(150);
 delay(2000);
 stopp();
 delay(2000);
 }
```

Dans la fonction loop(), le robot effectue une série de mouvements qui se répètent en boucle.
D'abord, on utilise la fonction go_forward(100) pour le faire avancer à vitesse moyenne. Ensuite, avec go_backward(100), il recule à cette même vitesse.
Après ça, le robot tourne sur lui-même vers la gauche avec la commande rotate_left(150), puis vers la droite avec rotate_right(150).

Enfin, on le stoppe complètement en appelant stopp().
Entre chaque action, il y a une pause de 2 secondes, grâce à delay(2000). Tout ce cycle recommence automatiquement sans fin tant que le robot est allumé.

La partie du code contenant la fonction **go_forward** :

```
void go_forward(unsigned char speed_val)     // speed_val: 0~255
    {digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,LOW);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,LOW);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);
```

La fonction go_forward() permet au robot d'avancer. Pour cela, elle active les deux moteurs dans le sens avant. Concrètement, elle met les broches arrière pinRB et pinLB à l'état haut (HIGH), ce qui lance les moteurs.
En même temps, les broches avant pinRF et pinLF sont mises à l'état bas (LOW), ce qui garantit que le mouvement se fait bien vers l'avant.
C'est ce réglage des broches qui détermine le sens de rotation des roues.

La partie du code contenant la fonction **go_backward** :

```
void go_backward(unsigned char speed_val)     // speed_val: 0~255
    {
     digitalWrite(pinRB,LOW);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,LOW);
     digitalWrite(pinLF,HIGH);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);
    }
```

La méthode go_backward() donne la capacité au robot de faire marche arrière. Elle change le sens de rotation des moteurs en comparaison à la fonction de progression. Les broches à l'arrière pinRB et pinLB sont configurées en LOW, alors que les broches à l'avant pinRF et pinLF sont configurées en HIGH. Cette inversion de polarité provoque une rotation des moteurs dans le sens opposé. Tout comme pour la progression, la vitesse est ajustée par le biais des signaux PWM transmis aux broches Lpwm_pin et Rpwm_pin.

La partie du code contenant les fonctions **rotate_left()** et **rotate_right()** :

```
void rotate_left(unsigned char speed_val)        // speed_val: 0~255
    {digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,LOW );
     digitalWrite(pinLB,LOW);
     digitalWrite(pinLF,HIGH);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);


    }
void rotate_right(unsigned char speed_val)    // speed_val: 0~255
    {
      digitalWrite(pinRB,LOW);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,LOW);
     analogWrite(Lpwm_pin,speed_val);
     analogWrite(Rpwm_pin,speed_val);

    }
void stopp()        //stop
    {
     digitalWrite(pinRB,HIGH);
     digitalWrite(pinRF,HIGH);
     digitalWrite(pinLB,HIGH);
     digitalWrite(pinLF,HIGH);
    }
```

Les méthodes rotate_left() et rotate_right() offrent la possibilité de faire tourner le robot sur son axe, vers la gauche ou vers la droite. Pour effectuer un virage à gauche, le moteur de droite est mis en marche avant pendant que le moteur de gauche est mis en marche arrière. En revanche, pour effectuer un virage à droite, le moteur gauche fonctionne en marche avant tandis que le moteur droit opère en marche arrière. Cette configuration entraîne une rotation sur place. Le contrôle de la vitesse de rotation est effectué par le PWM. Pour conclure, la fonction stopp() active toutes les broches de commande en mode HIGH, ce qui entraîne l'arrêt de l'approvisionnement en énergie des moteurs et l'immobilisation instantanée du robot.

Pour nous permettre de contrôler la **fonction détection d'obstacle**, nous avons utilisé le code ci-dessous :

```
#include <Servo.h>
Servo myservo;
int Echo_Pin=A0;  // ultrasonic module   ECHO to A0
int Trig_Pin=A1;  // ultrasonic module  TRIG to A1
#define Lpwm_pin  5     //pin of controlling speed---- ENA of motor driver board
#define Rpwm_pin  6    //pin of controlling speed---- ENB of motor driver board
int pinLB=2;             //pin of controlling turning---- IN1 of motor driver board
int pinLF=4;             //pin of controlling turning---- IN2 of motor driver board
int pinRB=7;            //pin of controlling turning---- IN3 of motor driver board
int pinRF=8;            //pin of controlling turning---- IN4 of motor driver board
volatile int D_mix;
volatile int D_mid;
volatile int D_max;
volatile int Front_Distance;
volatile int Left_Distance;
volatile int Right_Distance;
volatile int Right_IR_Value;
volatile int Left_IR_Value;
```

Le programme débute avec l'intégration de la bibliothèque Servo.h, qui offre la possibilité de contrôler un servomoteur. On crée un objet nommé myservo pour contrôler ce servo. Puis, on définit les broches associées aux capteurs ultrason (Echo sur A0 et Trig sur A1), ainsi qu'aux modules L298N (qui contrôlent la direction et la vitesse des moteurs).

La partie du code contenant les fonctions **checkdistance** :

```
float checkdistance() {
  digitalWrite(Trig_Pin, LOW);
  delayMicroseconds(2);
  digitalWrite(Trig_Pin, HIGH);
  delayMicroseconds(10);
  digitalWrite(Trig_Pin, LOW);
  float distance = pulseIn(Echo_Pin, HIGH) / 58.00;
  delay(10);
  return distance;
}
```

Cette fonctionnalité facilite la mesure d'une distance à l'aide du capteur à ultrason (HC-SR04). Elle déclenche une impulsion sur la broche Trig, puis évalue la durée nécessaire à l'onde pour revenir à la broche Echo. Cette durée est transformée en centimètres par le biais de la formule pulseIn(...) / 58.00. Ainsi, cette fonction renvoie une distance exprimée en centimètres, qui sera employée pour esquiver les obstacles.

La partie du code contenant la fonction **left_and_right:**

```
void Detect_Left_and_Right__distance() {
  myservo.write(180);
  delay(400);
  Left_Distance = checkdistance();
  delay(600);
  Serial.print("Left_Distance:");
  Serial.println(Left_Distance);
  myservo.write(0);
  delay(400);
  Right_Distance = checkdistance();
  delay(600);
  Serial.print("Right_Distance:");
  Serial.println(Right_Distance);
  myservo.write(90);
}
```

Cette fonctionnalité offre la capacité au robot d'évaluer les distances à gauche et à droite. Elle actionne le servomoteur jusqu'à 180° (à gauche) et 0° (à droite), positions dans lesquelles se fixe le capteur à ultrasons. Le programme appelle checkdistance() à chaque emplacement pour consigner la distance mesurée. Ces valeurs sont par la suite présentées sur le moniteur série en utilisant Serial.print(). Finalement, le capteur est remis à sa position centrale (90°).

La partie du code contenant la fonction **Ultrasonic_obstacle_avoidance()** :

```
void Ultrasonic_obstacle_avoidance()
{
  Front_Distance=checkdistance(); //obtain the value detected by ultrasonic sensor
  if((Front_Distance < 20)&&(Front_Distance > 0))//if the distance is greater than 0 and less than 20
{
    stopp();//stop
    delay(100);
    myservo.write(180);
    delay(500);
    Left_Distance=checkdistance();//measure the distance
    delay(100);
    myservo.write(0);
    delay(500);
    Right_Distance=checkdistance();//measure the distance
    delay(100);
if(Left_Distance > Right_Distance)//if distance a1 is greater than a2
    {
      rotate_left(150);//turn left
      myservo.write(90);
      delay(300);
    }
    else //if the right distance is greater than the left
    {
      rotate_right(150);// turn right
      myservo.write(90);
      delay(300);
    }
  }
  else//otherwise
  {
    go_forward(100);//go forward
  }
}
```

Cette fonctionnalité débute par l'évaluation de la distance frontale via le capteur. Si l'écart est moindre que 20 cm, le robot se stoppe avec stopp(), puis observe vers la gauche (180°) pour évaluer la distance, puis dirige à droite (0°) et procède à une nouvelle mesure. Si l'écart à gauche dépasse celui de droite, le robot pivote vers la gauche (rotate_left(150)), sinon il se tourne vers la droite (rotate_right(150)). Si aucun obstacle n'est présent devant lui, le robot poursuit simplement sa progression avec go_forward(100).

PARTIE 3- Code final

Voici les différentes parties du code que nous avons utilisé lors de l'évaluation pratique :

```
#include <Servo.h>

Servo myservo;
int Echo_Pin = A0;  // ultrasonic module   ECHO to A0
int Trig_Pin = A1;  // ultrasonic module  TRIG to A1
#define Lpwm_pin  5     //pin of controlling speed---- ENA of motor driver board
#define Rpwm_pin  6     //pin of controlling speed---- ENB of motor driver board
int pinLB = 2;       //pin of controlling turning---- IN1 of motor driver board
int pinLF = 4;       //pin of controlling turning---- IN2 of motor driver board
int pinRB = 7;       //pin of controlling turning---- IN3 of motor driver board
int pinRF = 8;       //pin of controlling turning---- IN4 of motor driver board

int D_mix;
int D_mid;
int D_max;
int Front_Distance;
int Left_Distance;
int Right_Distance;

float checkdistance() {
  digitalWrite(Trig_Pin, LOW);
  delayMicroseconds(2);
  digitalWrite(Trig_Pin, HIGH);
  delayMicroseconds(10);
  digitalWrite(Trig_Pin, LOW);
  long duration = pulseIn(Echo_Pin, HIGH, 30000);  // timeout 30ms
  if (duration == 0) return -1;  // no echo received
  float distance = duration / 58.0;
  delay(10);
  return distance;
}
```

Cette partie du code est consacrée à la fonction checkdistance(), laquelle vise à mesurer la distance entre le robot et un obstacle en se servant d'un capteur à ultrasons (type HC-SR04). Le processus implique l'envoi d'une impulsion ultrasonore via la broche Trig (activée par un signal HIGH de 10 microsecondes), suivi de la détermination du temps qu'il faut pour que cette onde soit renvoyée après avoir frappé un objet, en utilisant la broche Echo. La durée du vol est calculée à l'aide de la fonction pulseIn(), et cette période est ensuite convertie en distance (mesurée en centimètres) par division par 58, correspondant à la vitesse approximative du son dans l'air. Dans le cas où aucun écho n'est enregistré dans une durée de 30 ms.

```
void Detect_Left_and_Right_distance() {
  myservo.write(180);
  delay(400);
  Left_Distance = checkdistance();
  delay(600);
  Serial.print("Left_Distance: ");
  Serial.println(Left_Distance);

  myservo.write(0);
  delay(400);
  Right_Distance = checkdistance();
  delay(600);
  Serial.print("Right_Distance: ");
  Serial.println(Right_Distance);

  myservo.write(90);
}
```

Ce segment du code fait appel à la fonction Detect_Left_and_Right_distance(), qui utilise le servomoteur pour faire tourner le capteur à ultrasons vers la gauche et la droite afin de mesurer les distances dans ces orientations. Au départ, le servomoteur est orienté à 180° (vers la gauche), ensuite, une brève pause (delay(400)) est insérée afin de permettre au capteur de se stabiliser à cet endroit. Par la suite, on appelle la méthode checkdistance() pour mesurer et enregistrer la distance à gauche dans la variable Left_Distance. Cette information est ensuite affichée sur l'écran du moniteur série. On effectue ensuite une répétition à 0° (droite) pour mesurer la distance Right_Distance. Après avoir effectué ces deux opérations, le servomoteur est réinitialisé à 90° .

```
51  void Ultrasonic_obstacle_avoidance() {
52    Front_Distance = checkdistance(); //obtain the value detected by ultrasonic sensor
53    if ((Front_Distance < 20) && (Front_Distance > 0)) { //if the distance is greater than 0 and less than 20
54      stopp(); //stop
55      delay(100);
56
57      myservo.write(180);
58      delay(500);
59      Left_Distance = checkdistance();//measure the distance
60      delay(100);
61
62      myservo.write(0);
63      delay(500);
64      Right_Distance = checkdistance();//measure the distance
65      delay(100);
66
67      if (Left_Distance > Right_Distance) { //if left distance is greater than right
68        rotate_left(150);//turn left
69        myservo.write(90);
70        delay(300);
71      } else { //if the right distance is greater than the left
72        rotate_right(150);// turn right
73        myservo.write(90);
74        delay(300);
75      }
76    } else { //otherwise
77      go_forward(100);//go forward
78    }
79  }
80
```

La méthode Ultrasonic_obstacle_avoidance() commence par mesurer la distance devant le capteur grâce à la fonction checkdistance(), puis elle consigne cette valeur dans la variable Front_Distance. Si le robot détecte un obstacle à moins de 20 centimètres, il s'arrête immédiatement en appelant la fonction stopp(). Ensuite, via le servomoteur, le capteur est orienté vers la gauche (180°) pour mesurer la distance à gauche, puis vers la droite (0°) afin d'examiner celle à droite. Après avoir analysé les distances, le robot choisit le chemin qui offre le plus d'espace : s'il y a plus de place à sa gauche, il se tourne vers la gauche (rotate_left(150)), sinon il préfère la droite (rotate_right(150)). Si aucun obstacle n'est détecté devant lui, le robot continue simplement d'avancer en ligne droite grâce à la fonction go_forward(100). Ainsi, cette fonctionnalité offre au robot la capacité de faire des choix simples et réactifs pour prévenir les chocs.

```
void Obstacle_Avoidance_Main() {
  Ultrasonic_obstacle_avoidance();
}

void setup() {
  myservo.attach(A2);
  Serial.begin(9600);

  D_mix = 10;
  D_mid = 20;
  D_max = 100;
  Front_Distance = 0;
  Left_Distance = 0;
  Right_Distance = 0;

  myservo.write(90);

  pinMode(Echo_Pin, INPUT);
  pinMode(Trig_Pin, OUTPUT);
  pinMode(pinLB, OUTPUT); // pin 2
  pinMode(pinLF, OUTPUT); // pin 4
  pinMode(pinRB, OUTPUT); // pin 7
  pinMode(pinRF, OUTPUT); // pin 8
  pinMode(Lpwm_pin, OUTPUT); // pin 5 (PWM)
  pinMode(Rpwm_pin, OUTPUT); // pin 6 (PWM)
}
```

La première est extrêmement simple : elle utilise directement Ultrasonic_obstacle_avoidance(), ce qui permet une structure évidente au programme en séparant la logique principale d'évitement d'obstacles. Quant à la fonction setup(), elle est invoquée une seule fois au moment du démarrage du robot. Elle établit les composantes essentielles : le servomoteur est monté sur la broche A2 et réglé à une position médiane (90°), la communication série est lancée pour présenter les distances sur l'affichage série, et un certain nombre de variables (comme les distances relevées) sont initialisées à zéro. Ensuite, les broches liées au capteur à ultrasons et au module de contrôle des moteurs sont configurées en tant qu'entrées ou sorties selon leur application.

```
void loop() {
  Obstacle_Avoidance_Main();
}

void go_forward(unsigned char speed_val) {
  digitalWrite(pinRB, LOW);
  digitalWrite(pinRF, HIGH);
  digitalWrite(pinLB, LOW);
  digitalWrite(pinLF, HIGH);
  analogWrite(Lpwm_pin, speed_val);
  analogWrite(Rpwm_pin, speed_val);
}

void go_backward(unsigned char speed_val) {
  digitalWrite(pinRB, HIGH);
  digitalWrite(pinRF, LOW);
  digitalWrite(pinLB, HIGH);
  digitalWrite(pinLF, LOW);
  analogWrite(Lpwm_pin, speed_val);
  analogWrite(Rpwm_pin, speed_val);
}

void rotate_left(unsigned char speed_val) {
  digitalWrite(pinRB, HIGH);
  digitalWrite(pinRF, LOW);
  digitalWrite(pinLB, LOW);
  digitalWrite(pinLF, HIGH);
  analogWrite(Lpwm_pin, speed_val);
  analogWrite(Rpwm_pin, speed_val);
}

void rotate_right(unsigned char speed_val) {
  digitalWrite(pinRB, LOW);
  digitalWrite(pinRF, HIGH);
  digitalWrite(pinLB, HIGH);
  digitalWrite(pinLF, LOW);
  analogWrite(Lpwm_pin, speed_val);
  analogWrite(Rpwm_pin, speed_val);
}

void stopp() {
  analogWrite(Lpwm_pin, 0);
  analogWrite(Rpwm_pin, 0);
  digitalWrite(pinLB, LOW);
  digitalWrite(pinLF, LOW);
  digitalWrite(pinRB, LOW);
  digitalWrite(pinRF, LOW);
}
```

Cette section finale du code comprend la fonction loop() et toutes les fonctions qui régulent le mouvement du robot. La méthode loop() qui tourne sans interruption, fait appel à Obstacle_Avoidance_Main(), ce qui entraîne une détection et un contournement d'obstacles continus. Les méthodes go_forward(), go_backward(), rotate_left() et rotate_right() gèrent la direction et la vitesse du robot en transmettant des signaux aux broches de commande des moteurs à travers la carte de contrôle. Chaque fonction active les bons couples de broches pour faire tourner les roues dans le sens voulu, et emploie analogWrite() pour réguler la vitesse par le biais du signal PWM. En définitive, la fonction stopp() met fin aux moteurs en réduisant les signaux à 0, ce qui entraîne l'arrêt total du robot.

PARTIE 4 : Difficultés rencontrées

Tout au long de la réalisation du robot, nous avons rencontré plusieurs difficultés techniques que nous avons su surmonter étape par étape.

Premièrement, nous avons mal positionné le servomoteur sur le châssis, ce qui a empêché l'installation correcte des moteurs. Cela nous a obligés à démonter plusieurs composants, dont le L298N, pour repositionner le servomoteur correctement.

Ensuite, une erreur dans le code a provoqué un comportement inattendu : le robot avançait en marche arrière. Après correction, un nouveau problème est apparu au niveau des roues: l'une avançait plus vite que l'autre. En analysant la situation, nous avons constaté que les roues tournaient dans des directions opposées. Après avoir corrigé leur orientation, le robot roulait mieux, mais un déséquilibre persistait.

Nous avons alors découvert que le moteur droit n'était pas alimenté, en raison d'une soudure mal réalisée. Une fois le moteur ressoudé, le problème persistait. Nous avons finalement identifié un mauvais contact à la masse, ce qui cause un court-circuit. En resserrant correctement les connexions, le contact avec la masse a été rétabli.
Grâce à ces ajustements successifs, notre robot a pu fonctionner correctement et de manière autonome.

PARTIE 5 : Conclusion du projet

Au terme de ce SAE, nous avons réussi à concevoir, programmer et tester un robot autonome qui peut éviter les obstacles grâce à un système intégrant un capteur ultrason, un servomoteur pour la détection et un module de pilotage des moteurs.

En dépit de divers défis techniques, comme des soucis de câblage, l'absence de certains équipements ou l'impossibilité d'accéder à quelques ordinateurs pour le transfert du code, nous avons démontré notre capacité à nous adapter et notre persistance. Le code final, basé sur une stratégie d'évitement d'obstacles, donne au robot la capacité d'évaluer les distances en face de lui, à gauche et à droite pour décider du chemin à emprunter.

Ce projet a mis à l'épreuve nos aptitudes en programmation Arduino, en électronique et en travail d'équipe.

Remerciement

Nous tenons à remercier Monsieur Traore pour son accompagnement tout au long de ce projet. Ses conseils, sa disponibilité et son encadrement nous ont permis de mieux comprendre les étapes de réalisation du robot et d'approfondir nos connaissances pratique.